Project: Diamond Price Analysis

To analyze the price of diamonds according to their attributes, we first need to have a dataset containing diamond prices based on their features

1. Carat
2. Cut
3. Colour
4. Clarity
5. Depth
6. Table
7. Price
8. Size Dataset: https://drive.google.com/file/d/1I4wk5hVhl8l2Jl7sNbOq3du96CBZyTEq/view?usp=sharing

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
```

```
import pandas as pd
# Load dataset
df = pd.read_csv('diamonds.csv')
df = df.drop(columns=['Unnamed: 0'])
```

```
# Create new feature: size = x * y * z
df['size'] = df['x'] * df['y'] * df['z']
```

```
# Features and target
X = df[['carat', 'cut', 'color', 'clarity', 'depth', 'table', 'size']]
y = df['price']
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Column types
categorical_features = ['cut', 'color', 'clarity']
numerical_features = ['carat', 'depth', 'table', 'size']
```

```
df=pd.read_csv("diamonds.csv")
df=df.drop(columns=['Unnamed: 0'])
df['size']=df['x']*df['y']*df['z']
df=df.sample(n=10000, random_state=42)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 10000 entries, 1388 to 42497
Data columns (total 11 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   carat    10000 non-null  float64
 1   cut      10000 non-null  object
 2   color    10000 non-null  object
 3   clarity  10000 non-null  object
 4   depth    10000 non-null  float64
 5   table    10000 non-null  float64
 6   price    10000 non-null  int64
 7   x        10000 non-null  float64
 8   y        10000 non-null  float64
 9   z        10000 non-null  float64
 10  size     10000 non-null  float64
dtypes: float64(7), int64(1), object(3)
memory usage: 937.5+ KB
```

```
df.dtypes
```

	0
carat	float64
cut	object
color	object
clarity	object
depth	float64
table	float64
price	int64
x	float64
y	float64
z	float64
size	float64

dtype: object

```
df.head(10)
```

	carat	cut	color	clarity	depth	table	price	x	y	z	size
1388	0.24	Ideal	G	VVS1	62.1	56.0	559	3.97	4.00	2.47	39.223600
50052	0.58	Very Good	F	VVS2	60.0	57.0	2201	5.44	5.42	3.26	96.120448
41645	0.40	Ideal	E	VVS2	62.1	55.0	1238	4.76	4.74	2.95	66.559080
42377	0.43	Premium	E	VVS2	60.8	57.0	1304	4.92	4.89	2.98	71.695224
17244	1.55	Ideal	E	SI2	62.3	55.0	6901	7.44	7.37	4.61	252.779208
1608	1.00	Fair	E	SI2	55.4	62.0	3011	6.63	6.59	3.66	159.911622
46398	0.51	Ideal	F	VS1	60.2	56.0	1765	5.22	5.24	3.15	86.161320
45493	0.52	Ideal	D	VS2	62.0	56.0	1679	5.17	5.19	3.21	86.131683
49385	0.62	Premium	E	VS2	60.0	59.0	2102	5.58	5.56	3.34	103.622832
10460	1.14	Ideal	H	SI1	60.3	57.0	4789	6.79	6.85	4.11	191.162265

Next steps: Generate code with df New interactive sheet

```
df.tail(10)
```

	carat	cut	color	clarity	depth	table	price	x	y	z	size
20744	1.05	Ideal	E	VVS2	62.4	56.0	8982	6.49	6.52	4.06	171.798088
46159	0.50	Ideal	E	VS2	60.0	57.0	1746	5.15	5.12	3.08	81.213440
32676	0.31	Premium	G	VS1	61.8	59.0	802	4.35	4.32	2.68	50.362560
12604	1.01	Premium	G	VS2	60.9	55.0	5288	6.50	6.48	3.95	166.374000
6969	1.01	Good	G	SI2	62.4	60.0	4150	6.32	6.38	3.96	159.673536
33818	0.40	Premium	F	VS2	61.6	58.0	842	4.75	4.80	2.94	67.032000
10197	0.91	Very Good	F	VS1	63.5	56.0	4739	6.18	6.14	3.91	148.365732
34821	0.30	Very Good	H	VVS1	63.3	55.0	878	4.34	4.29	2.73	50.828778
31584	0.35	Ideal	E	VS2	62.5	55.0	767	4.50	4.52	2.82	57.358800
42497	0.54	Very Good	E	SI1	62.2	61.0	1320	5.24	5.15	3.23	87.164780

```
df.isna().sum()
```

	0
carat	0
cut	0
color	0
clarity	0
depth	0
table	0
price	0
x	0
y	0
z	0
size	0

dtype: int64

```
df.describe(include='all')
```

	carat	cut	color	clarity	depth	table	price	x	y	z
count	10000.000000	10000	10000	10000	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
unique	NaN	5	7	8	NaN	NaN	NaN	NaN	NaN	NaN
top	NaN	Ideal	G	SI1	NaN	NaN	NaN	NaN	NaN	NaN
freq	NaN	3980	2105	2360	NaN	NaN	NaN	NaN	NaN	NaN
mean	0.795464	NaN	NaN	NaN	61.77341	57.447340	3900.356600	5.722033	5.723494	3.534455
std	0.476937	NaN	NaN	NaN	1.45393	2.255054	3992.251932	1.128186	1.120839	0.696424
min	0.200000	NaN	NaN	NaN	43.00000	49.000000	335.000000	0.000000	0.000000	0.000000
25%	0.400000	NaN	NaN	NaN	61.10000	56.000000	929.750000	4.690000	4.700000	2.900000
50%	0.700000	NaN	NaN	NaN	61.90000	57.000000	2385.500000	5.690000	5.700000	3.520000
75%	1.040000	NaN	NaN	NaN	62.50000	59.000000	5235.250000	6.530000	6.530000	4.030000
max	4.010000	NaN	NaN	NaN	79.00000	73.000000	18823.000000	10.140000	10.100000	6.310000

```
df.duplicated().value_counts()
```

	count
False	9995
True	5

dtype: int64

```
print("DiAMOND Dataset consist of {rows}.".format(rows=len(df)))
```

```
DiAMOND Dataset consist of 10000.
```

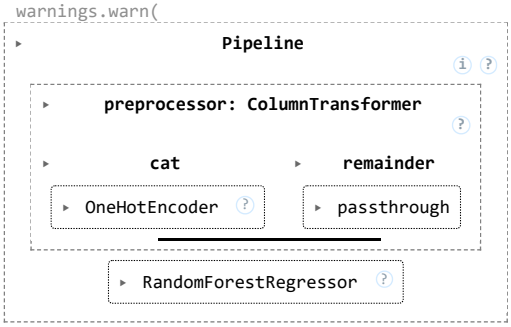
```
# Preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough'
)
```

```
#Machine learning pipeline
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=100, random_state=42))
])
```

```
# Train model
model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/compose/_column_transformer.py:1667: FutureWarning:
The format of the columns of the 'remainder' transformer in ColumnTransformer.transformers_ will change in version 1.7 to ma
At the moment the remainder columns are stored as indices (of type int). With the same ColumnTransformer configuration, in t
To use the new behavior now and suppress this warning, use ColumnTransformer(force_int_remainder_cols=False).

  warnings.warn(
```




```
# Predict
y_pred = model.predict(X_test)
```

```
# Evaluation
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print(f"Mean Absolute Error: ${mae:.2f}")
print(f"R^2 Score: {r2:.3f}")
```

```
Mean Absolute Error: $279.53
R^2 Score: 0.981
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

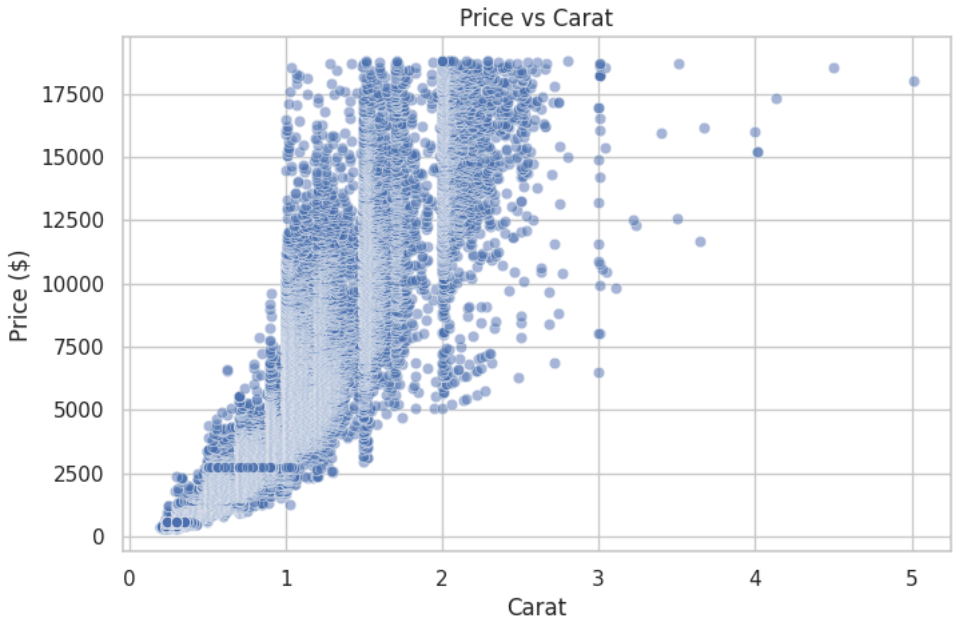
```
# Load dataset
df = pd.read_csv('diamonds.csv')
```

```
# Drop unnecessary column
df = df.drop(columns=['Unnamed: 0'])
```

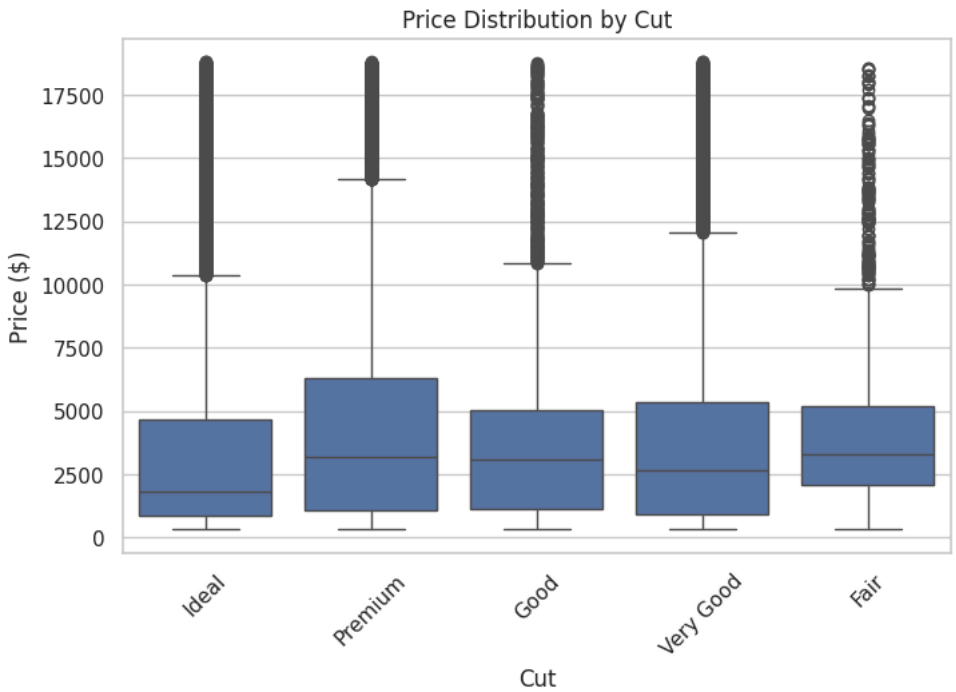
```
# Create 'size' feature as volume
df['size'] = df['x'] * df['y'] * df['z']
```

```
# Set Seaborn style
sns.set(style="whitegrid")
```

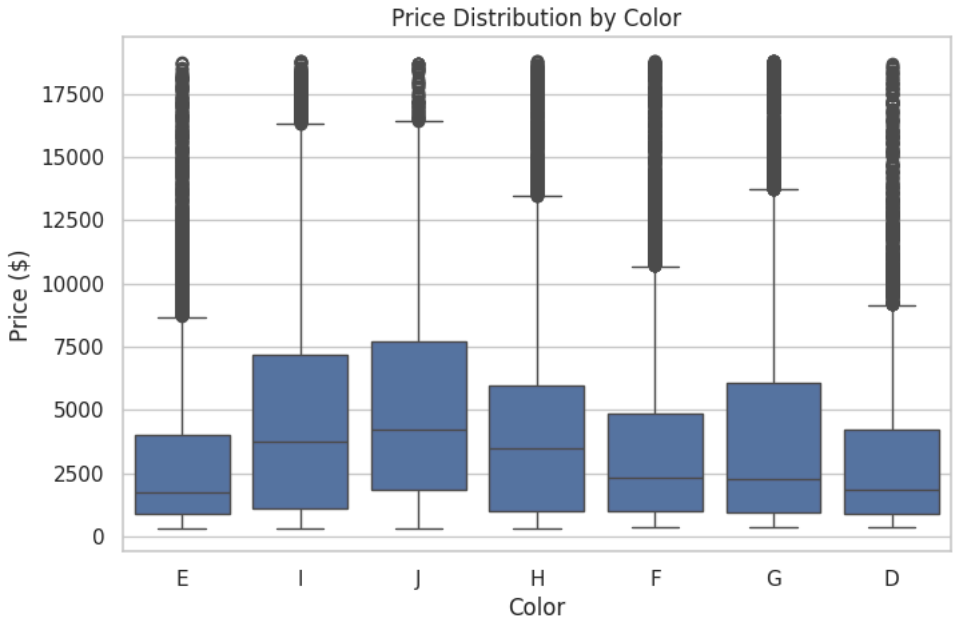
```
# Price vs Carat
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='carat', y='price', alpha=0.5)
plt.title('Price vs Carat')
plt.xlabel('Carat')
plt.ylabel('Price ($)')
plt.show()
```



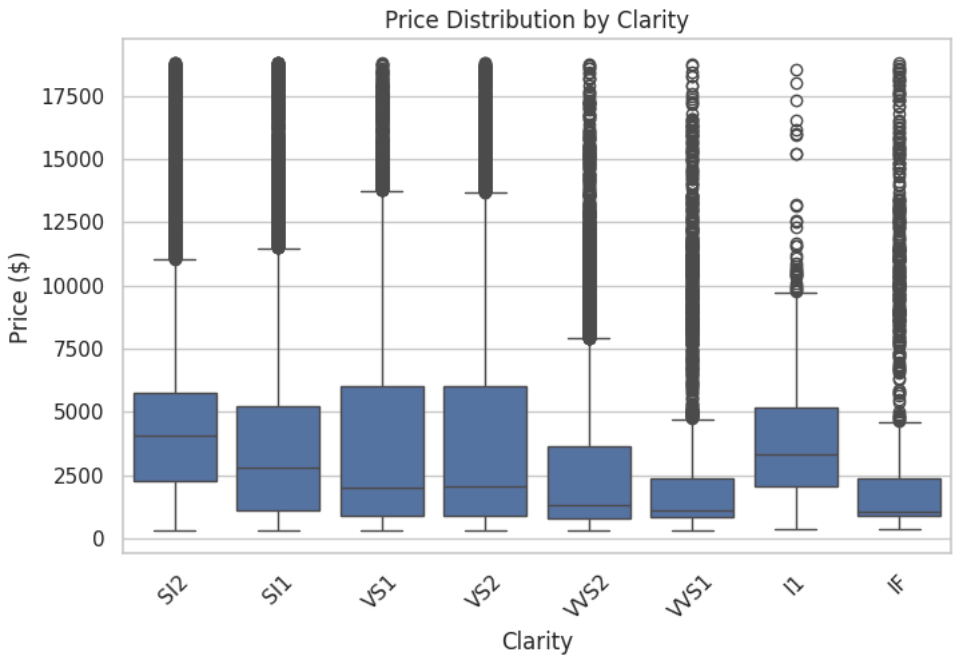

```
# Boxplot: Price by Cut
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='cut', y='price')
plt.title('Price Distribution by Cut')
plt.xlabel('Cut')
plt.ylabel('Price ($)')
plt.xticks(rotation=45)
plt.show()
```



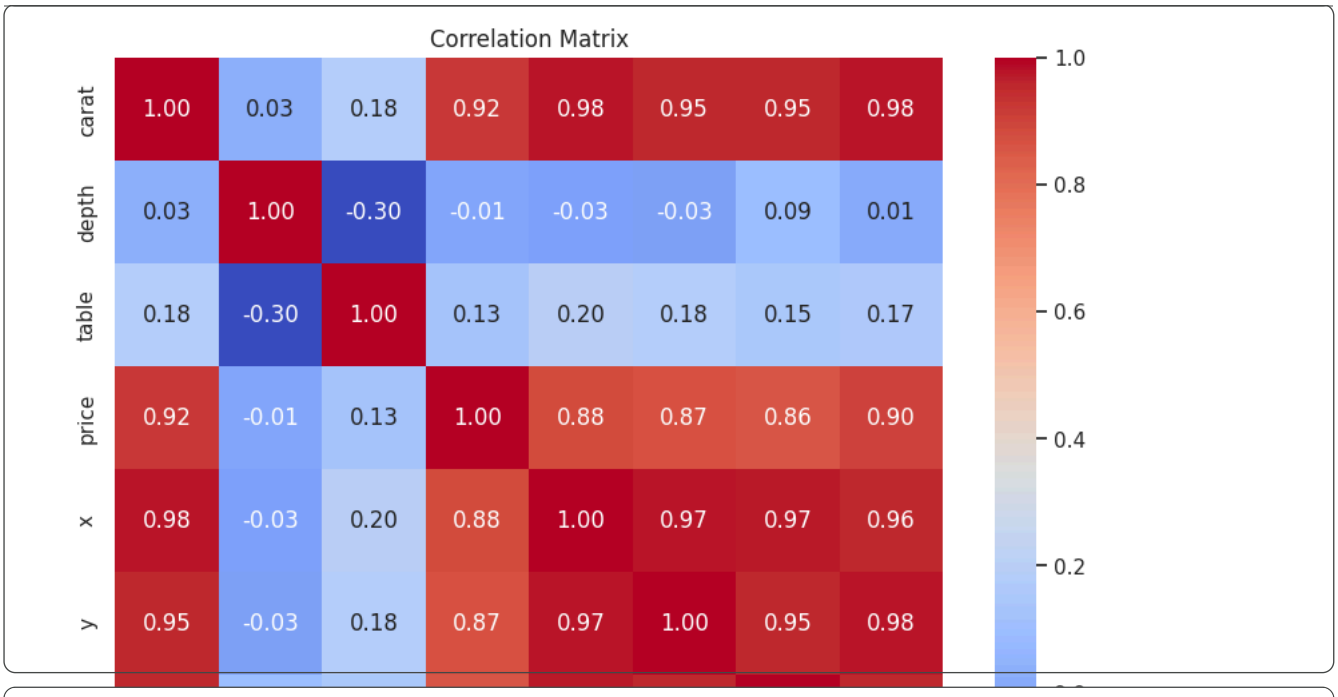

```
# Boxplot: Price by Color
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='color', y='price')
plt.title('Price Distribution by Color')
plt.xlabel('Color')
plt.ylabel('Price ($)')
plt.show()
```




```
# Boxplot: Price by Clarity
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='clarity', y='price')
plt.title('Price Distribution by Clarity')
plt.xlabel('Clarity')
plt.ylabel('Price ($)')
plt.xticks(rotation=45)
plt.show()
```




```
# Correlation heatmap
plt.figure(figsize=(10, 8))
corr = df[['carat', 'depth', 'table', 'price', 'x', 'y', 'z', 'size']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.show()
```




```
#USER INPUT
carat = float(input("Enter Carat: "))
cut = input("Enter Cut (Fair, Good, Very Good, Premium, Ideal): ")
color = input("Enter Color (D, E, F, G, H, I, J): ")
clarity = input("Enter Clarity (I1, SI2, SI1, VS2, VS1, VVS2, VVS1, IF): ")
depth = float(input("Enter Depth: "))
table = float(input("Enter Table: "))
x = float(input("Enter Length (x): "))
y = float(input("Enter Width (y): "))
z = float(input("Enter Height (z): "))

#CREATE SIZE FEATURE
size = x * y * z

#CREATE INPUT DATAFRAME
import pandas as pd

input_data = pd.DataFrame({
    'carat': [carat],
```